VLLM专题（二十五）—使用统计数据收集

vLLM 默认收集匿名的使用数据，以帮助工程团队更好地了解哪些硬件和模型配置被广泛使用。这些数据使他们能够将精力集中在最常见的工作负载上。收集的数据是透明的，不包含任何敏感信息，并且将公开发布，以供社区使用。

1. 收集了哪些数据？

最新版本的 vLLM 收集的数据列表可以在如下代码找到：`usage_lib.py`

```python
# SPDX-License-Identifier: Apache-2.0

import datetime
import json
import logging
import os
import platform
import time
from enum import Enum
from pathlib import Path
from threading import Thread
from typing import Any, Optional, Union
from uuid import uuid4

import cpuinfo
import psutil
import requests
import torch

import vllm.envs as envs
from vllm.connections import global_http_connection
from vllm.version import __version__ as VLLM_VERSION

_config_home = envs.VLLM_CONFIG_ROOT
_USAGE_STATS_JSON_PATH = os.path.join(_config_home, "usage_stats.json")
_USAGE_STATS_DO_NOT_TRACK_PATH = os.path.join(_config_home, "do_not_track")
_USAGE_STATS_ENABLED = None
_USAGE_STATS_SERVER = envs.VLLM_USAGE_STATS_SERVER
```

```python
_GLOBAL_RUNTIME_DATA = dict[str, Union[str, int, bool]]()

_USAGE_ENV_VARS_TO_COLLECT = [
    "VLLM_USE_MODELSCOPE",
    "VLLM_USE_TRITON_FLASH_ATTN",
    "VLLM_ATTENTION_BACKEND",
    "VLLM_USE_FLASHINFER_SAMPLER",
    "VLLM_PP_LAYER_PARTITION",
    "VLLM_USE_TRITON_AWQ",
    "VLLM_USE_V1",
    "VLLM_ENABLE_V1_MULTIPROCESSING",
]


def set_runtime_usage_data(key: str, value: Union[str, int, bool]) -> None:
    """Set global usage data that will be sent with every usage heartbeat."""
    _GLOBAL_RUNTIME_DATA[key] = value


def is_usage_stats_enabled():
    """Determine whether or not we can send usage stats to the server.
    The logic is as follows:
    - By default, it should be enabled.
    - Three environment variables can disable it:
        - VLLM_DO_NOT_TRACK=1
        - DO_NOT_TRACK=1
        - VLLM_NO_USAGE_STATS=1
    - A file in the home directory can disable it if it exists:
        - $HOME/.config/vllm/do_not_track
    """
    global _USAGE_STATS_ENABLED
    if _USAGE_STATS_ENABLED is None:
        do_not_track = envs.VLLM_DO_NOT_TRACK
        no_usage_stats = envs.VLLM_NO_USAGE_STATS
        do_not_track_file = os.path.exists(_USAGE_STATS_DO_NOT_TRACK_PATH)

        _USAGE_STATS_ENABLED = not (do_not_track or no_usage_stats
                                    or do_not_track_file)
    return _USAGE_STATS_ENABLED


def _get_current_timestamp_ns() -> int:
    return int(datetime.datetime.now(datetime.timezone.utc).timestamp() * 1e9)


def _detect_cloud_provider() -> str:
```

```
    # Try detecting through vendor file
    vendor_files = [
        "/sys/class/dmi/id/product_version", "/sys/class/dmi/id/bios_vendor",
        "/sys/class/dmi/id/product_name",
        "/sys/class/dmi/id/chassis_asset_tag", "/sys/class/dmi/id/sys_vendor"
    ]
    # Mapping of identifiable strings to cloud providers
    cloud_identifiers = {

        "amazon": "AWS",
        "microsoft corporation": "AZURE",
        "google": "GCP",
        "oraclecloud": "OCI",
    }

    for vendor_file in vendor_files:
        path = Path(vendor_file)
        if path.is_file():
            file_content = path.read_text().lower()
            for identifier, provider in cloud_identifiers.items():
                if identifier in file_content:
                    return provider

    # Try detecting through environment variables
    env_to_cloud_provider = {

        "RUNPOD_DC_ID": "RUNPOD",
    }
    for env_var, provider in env_to_cloud_provider.items():
        if os.environ.get(env_var):
            return provider

    return "UNKNOWN"


class UsageContext(str, Enum):
    UNKNOWN_CONTEXT = "UNKNOWN_CONTEXT"
    LLM_CLASS = "LLM_CLASS"
    API_SERVER = "API_SERVER"
    OPENAI_API_SERVER = "OPENAI_API_SERVER"
    OPENAI_BATCH_RUNNER = "OPENAI_BATCH_RUNNER"
    ENGINE_CONTEXT = "ENGINE_CONTEXT"


class UsageMessage:
    """Collect platform information and send it to the usage stats server."""

    def __init__(self) -> None:
```

```
    # NOTE: vLLM's server _only_ support flat KV pair.
    # Do not use nested fields.

    self.uuid = str(uuid4())

    # Environment Information
    self.provider: Optional[str] = None
    self.num_cpu: Optional[int] = None
    self.cpu_type: Optional[str] = None
    self.cpu_family_model_stepping: Optional[str] = None
    self.total_memory: Optional[int] = None
    self.architecture: Optional[str] = None
    self.platform: Optional[str] = None
    self.cuda_runtime: Optional[str] = None
    self.gpu_count: Optional[int] = None
    self.gpu_type: Optional[str] = None
    self.gpu_memory_per_device: Optional[int] = None
    self.env_var_json: Optional[str] = None

    # vLLM Information
    self.model_architecture: Optional[str] = None
    self.vllm_version: Optional[str] = None
    self.context: Optional[str] = None

    # Metadata
    self.log_time: Optional[int] = None
    self.source: Optional[str] = None

def report_usage(self,
                 model_architecture: str,
                 usage_context: UsageContext,
                 extra_kvs: Optional[dict[str, Any]] = None) -> None:
    t = Thread(target=self._report_usage_worker,
               args=(model_architecture, usage_context, extra_kvs or {
        }),
               daemon=True)
    t.start()

def _report_usage_worker(self, model_architecture: str,
                         usage_context: UsageContext,
                         extra_kvs: dict[str, Any]) -> None:
    self._report_usage_once(model_architecture, usage_context, extra_kvs)
    self._report_continous_usage()

def _report_usage_once(self, model_architecture: str,
                       usage_context: UsageContext,
                       extra_kvs: dict[str, Any]) -> None:
```

```
        # Platform information
        from vllm.platforms import current_platform
        if current_platform.is_cuda_alike():
            device_property = torch.cuda.get_device_properties(0)
            self.gpu_count = torch.cuda.device_count()
            self.gpu_type = device_property.name
            self.gpu_memory_per_device = device_property.total_memory
        if current_platform.is_cuda():
            self.cuda_runtime = torch.version.cuda
        self.provider = _detect_cloud_provider()
        self.architecture = platform.machine()
        self.platform = platform.platform()
        self.total_memory = psutil.virtual_memory().total

        info = cpuinfo.get_cpu_info()
        self.num_cpu = info.get("count", None)
        self.cpu_type = info.get("brand_raw", "")
        self.cpu_family_model_stepping = ",".join([
            str(info.get("family", "")),
            str(info.get("model", "")),
            str(info.get("stepping", ""))
        ])

        # vLLM information
        self.context = usage_context.value
        self.vllm_version = VLLM_VERSION
        self.model_architecture = model_architecture

        # Environment variables
        self.env_var_json = json.dumps({

            env_var: getattr(envs, env_var)
            for env_var in _USAGE_ENV_VARS_TO_COLLECT
        })

        # Metadata
        self.log_time = _get_current_timestamp_ns()
        self.source = envs.VLLM_USAGE_SOURCE

        data = vars(self)
        if extra_kvs:
            data.update(extra_kvs)

        self._write_to_file(data)
        self._send_to_server(data)

    def _report_continous_usage(self):
```

```
        """Report usage every 10 minutes.

        This helps us to collect more data points for uptime of vLLM usages.
        This function can also help send over performance metrics over time.
        """
        while True:
            time.sleep(600)
            data = {

                "uuid": self.uuid,
                "log_time": _get_current_timestamp_ns(),
            }
            data.update(_GLOBAL_RUNTIME_DATA)

            self._write_to_file(data)
            self._send_to_server(data)

    def _send_to_server(self, data: dict[str, Any]) -> None:
        try:
            global_http_client = global_http_connection.get_sync_client()
            global_http_client.post(_USAGE_STATS_SERVER, json=data)
        except requests.exceptions.RequestException:
            # silently ignore unless we are using debug log
            logging.debug("Failed to send usage data to server")

    def _write_to_file(self, data: dict[str, Any]) -> None:
        os.makedirs(os.path.dirname(_USAGE_STATS_JSON_PATH), exist_ok=True)
        Path(_USAGE_STATS_JSON_PATH).touch(exist_ok=True)
        with open(_USAGE_STATS_JSON_PATH, "a") as f:
            json.dump(data, f)
            f.write("\n")


usage_message = UsageMessage()
```

以下是 v0.4.0 版本的示例：

```
{

  "uuid": "fbe880e9-084d-4cab-a395-8984c50f1109",
  "provider": "GCP",
  "num_cpu": 24,
  "cpu_type": "Intel(R) Xeon(R) CPU @ 2.20GHz",
  "cpu_family_model_stepping": "6,85,7",
  "total_memory": 101261135872,
  "architecture": "x86_64",
  "platform": "Linux-5.10.0-28-cloud-amd64-x86_64-with-glibc2.31",
  "gpu_count": 2,
  "gpu_type": "NVIDIA L4",
  "gpu_memory_per_device": 23580639232,
  "model_architecture": "OPTForCausalLM",
  "vllm_version": "0.3.2+cu123",
  "context": "LLM_CLASS",
  "log_time": 1711663373492490000,
  "source": "production",
  "dtype": "torch.float16",
  "tensor_parallel_size": 1,
  "block_size": 16,
  "gpu_memory_utilization": 0.9,
  "quantization": null,
  "kv_cache_dtype": "auto",
  "enable_lora": false,
  "enable_prefix_caching": false,
  "enforce_eager": false,
  "disable_custom_all_reduce": true
}
```


您可以通过运行以下命令预览收集的数据：

```
tail ~/.config/vllm/usage_stats.json
```

2. 选择退出

您可以通过设置 `VLLM_NO_USAGE_STATS` 或 `DO_NOT_TRACK` 环境变量，或通过创建 `~/.config/vllm/do_not_track` 文件来选择退出使用统计数据的收集：

```
# Any of the following methods can disable usage stats collection
export VLLM_NO_USAGE_STATS=1
export DO_NOT_TRACK=1
mkdir -p ~/.config/vllm && touch ~/.config/vllm/do_not_track
```